

kontx

Understand, audit and document any codebase.

Yours. Your team's. Or the one AI wrote for you.

npm v1.0.0 downloads package not found or too new license repo not found node 20+

static analysis → structured context → AI explanation

What is kontx?

kontx is a CLI tool for understanding, auditing, reviewing and documenting codebases with AI.

Instead of throwing an unfamiliar project at an AI model and asking it to figure everything out, kontx analyzes the codebase first.

It extracts structural information such as:

- Imports and dependencies
- Symbols and exports
- Environment variables
- Routes
- Project structure
- Test coverage signals
- Security signals
- Git changes

It then gives those facts — alongside the relevant source code — to the AI model you choose.

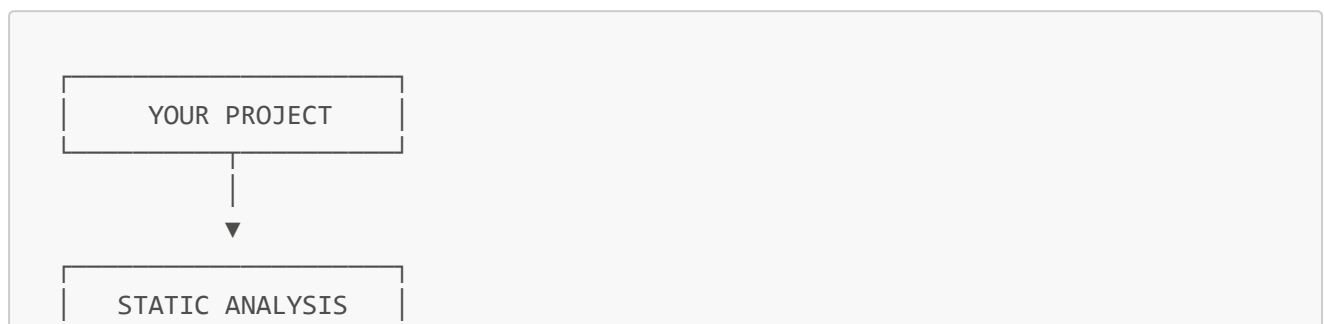
The code tells kontx what exists.

The AI explains what it means.

Why kontx?

Most AI code tools start with the model.

kontx starts with the **code**.





This separation matters.

Static analysis is deterministic.

AI is probabilistic.

Use each for what it is good at.

Install

```
npm install -g @reabot6/kontx
```

Requires **Node.js 20+**.

Verify the installation:

```
kontx --help
```

Quick start

Configure your AI provider:

```
kontx config
```

Then run kontx from your project:

```
cd my-project
```

```
kontx full
```

Or run a specific analysis:

```
kontx risk
kontx bug
kontx qa
kontx env
kontx readme
kontx diff
```

Generated output is written to:

```
.kontx/
```

Commands

Command	What it does
full	Full codebase audit — architecture, dependencies, environment, tests and risk
risk	Security and quality risk report
readme	Generate or refresh <code>README.md</code>
qa	Find coverage gaps, edge cases and missing test assertions

Command	What it does
<code>bug</code>	Hunt for likely bugs and logic errors
<code>env</code>	Audit environment variables and their usage
<code>diff</code>	Review staged changes or a branch diff
<code>stack</code>	Document the technology stack and architecture
<code>flow <file></code>	Trace execution flow from a file
<code>add <feature></code>	Plan how to add a feature
<code>mix <a> </code>	Compare two files
<code>config</code>	Configure your AI provider
<code>models</code>	List models available with your current key
<code>cache</code>	View cache statistics

Examples

Full audit

```
kontx full
```

Get a complete overview of an unfamiliar project.

Security audit

```
kontx risk
```

Find potential security vulnerabilities and quality risks.

Find bugs

```
kontx bug
```

Ask the configured AI to investigate likely bugs and logic errors using kontx's project context.

Audit environment variables

```
kontx env
```

Understand which environment variables your application uses and where.

Review a Git diff

```
kontx diff
```

Review staged changes or compare a branch.

```
kontx diff --base main
```

Trace execution flow

```
kontx flow src/index.js
```

Follow the execution path beginning from a specific file.

Plan a feature

```
kontx add 'add rate limiting to all routes'
```

Get a step-by-step implementation plan based on the existing codebase.

Compare files

```
kontx mix src/auth.js src/middleware.js
```

Compare two files with the context of the surrounding project.

AI providers

kontx is designed to work with the AI provider you prefer.

Supported providers include:

- Anthropic
- OpenAI
- Google Gemini
- Groq
- OpenRouter
- Ollama
- Mistral
- DeepSeek
- xAI
- Fireworks
- Cerebras
- Other OpenAI-compatible endpoints

You can configure your provider once:

```
kontx config
```

Or override it for an individual command.

Anthropic

```
ANTHROPIC_API_KEY=sk-ant-... kontx full
```

OpenAI

```
OPENAI_API_KEY=sk-... kontx full --model gpt-4o
```

Gemini

```
GEMINI_API_KEY=AIza... kontx full --model gemini-2.5-pro
```

Groq

```
GROQ_API_KEY=gsk-... kontx full --model llama-3.3-70b-versatile
```

Ollama

Run against a local model:

```
kontx full --provider ollama --model llama3.2
```

No API key required.

Any OpenAI-compatible endpoint

```
kontx full \  
  --base-url https://your-host/v1 \  
  --key <key> \  
  --model your-model
```

This makes kontx compatible with self-hosted inference servers and other OpenAI-compatible APIs.

Options

Global options are available across commands:

```
--model <id>  
  Override the model.  
  
--provider <name>  
  Force an AI provider.  
  
--key <key>  
  API key for this run.  
  Not saved.  
  
--base-url <url>  
  Custom OpenAI-compatible base URL.  
  
--budget <tokens>  
  Context token budget.  
  Default: 60000  
  
--max-output <n>  
  Maximum output tokens per request.  
  Default: 8000
```

```
--timeout <secs>
    Request timeout.
    Default: 300

--exclude <glob>
    Additional ignore pattern.
    Can be repeated.

--output <file>
    Write output to a custom path
    instead of .kontx/.

--fresh
    Ignore the cache and make a new request.

--no-ai
    Run static analysis only.

--dry-run
    Show what would be sent without
    making an AI request.

--yes
    Skip the consent prompt.
    Useful for CI and scripts.

--no-color
    Disable colour output.
```

Safety first

kontx is designed around a simple rule:

You should know what is being sent to an AI before it leaves your machine.

Before an AI request, kontx processes the project locally and applies its filtering and redaction rules.

You can inspect the request with:

```
kontx full --dry-run
```

For automated workflows where confirmation is not possible:

```
kontx risk --yes
```

What gets sent?

kontx works with source code and project information needed for the selected analysis.

Files are filtered according to kontx's ignore and safety rules.

Sensitive values are redacted before transmission.

By default, sensitive files such as:

```
.env
*.pem
*.key
service-account.json
credentials.*
```

are excluded.

Files matching your project's ignore rules can also be excluded.

You can add additional exclusions with:

```
kontx full --exclude "path/to/**"
```

Or create a:

```
.kontxignore
```

file in your project.

.kontxignore

.kontxignore uses gitignore-style patterns.

Example:

```
# Internal documentation
docs/internal/**

# Generated files
generated/**
```

```
# Large datasets
data/**

# Local experiments
experiments/**
```

This gives you explicit control over which parts of your project kontx can process.

Dry runs

Before making an AI request:

```
kontx full --dry-run
```

Use this when you want to inspect the analysis without sending anything.

This is especially useful when working with:

- Private repositories
- Client projects
- Proprietary code
- Internal tools
- Sensitive infrastructure

--no-ai

kontx does not require an AI call for its static analysis layer.

Run:

```
kontx full --no-ai
```

This allows you to inspect the deterministic project analysis without requesting an AI-generated report.

Caching

AI requests can get expensive.

kontx caches results so unchanged work does not need to be regenerated.

The cache is stored under:

```
.kontx/cache/
```

Check cache statistics:

```
kontx cache
```

Clear the cache:

```
kontx cache --clear
```

Force a fresh request:

```
kontx full --fresh
```

The goal is simple:

```
same input
  ↓
same analysis
  ↓
cache hit
  ↓
no unnecessary AI request
```

Project output

kontx keeps its generated data separate from your source code:

```
my-project/
├──
├── src/
├── tests/
├── package.json
├── README.md
├──
└── .kontx/
```

```
└─ cache/  
└─ ...
```

The `.kontx/` directory is intended for kontx-generated artifacts and should not need to become part of your application source.

Configuration

Run:

```
kontx config
```

to configure your provider and model.

Configuration is stored locally.

You can also avoid persistent credentials by using environment variables or:

```
kontx full --key <key>
```

The `--key` option applies only to that invocation.

CI

kontx can be used in automated workflows.

For example:

```
name: kontx  
  
on:  
  pull_request:  
  push:  
    branches:  
      - main  
  
jobs:  
  risk:  
    runs-on: ubuntu-latest  
  
    steps:  
      - uses: actions/checkout@v4
```

```
- uses: actions/setup-node@v4
  with:
    node-version: 20

- run: npx @reabot6/kontx risk --yes
  env:
    ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
```

The same pattern can be used for:

```
kontx full --yes
kontx risk --yes
kontx qa --yes
kontx diff --yes
```

Models

See the models available through your configured provider:

```
kontx models
```

You can override the model for a single command:

```
kontx risk --model gemini-2.5-pro
```

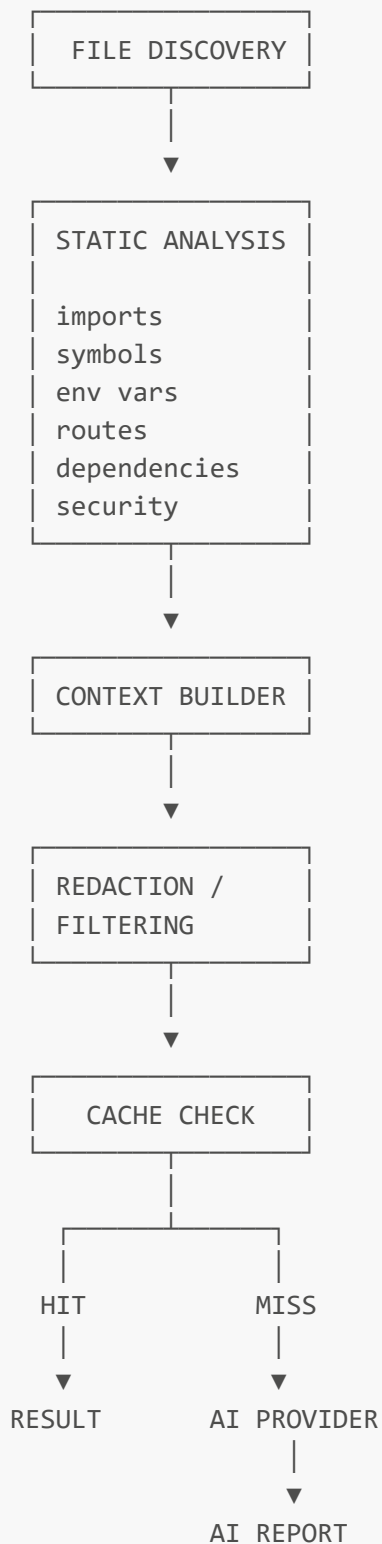
Or:

```
kontx full --model claude-sonnet-5-5
```

Architecture

kontx separates **code discovery** from **AI reasoning**.

```
CODEBASE
|
▼
```



The model receives structured context instead of being responsible for discovering the entire project from scratch.

Philosophy

AI is powerful.

But asking an AI model to discover everything about a large codebase from raw source is an inefficient way to use that power.

kontx takes a different approach.

```
CODE
↓
FACTS
↓
CONTEXT
↓
AI
↓
UNDERSTANDING
```

Let deterministic systems discover what can be computed.

Let AI reason about what requires interpretation.

Version 1

v1.0.0

This is the first public version of kontx.

The v1 foundation includes:

- Codebase static analysis
- AI-powered project understanding
- Multiple AI providers
- Security and quality analysis
- Environment auditing
- Bug analysis
- QA analysis
- Git diff review
- Architecture documentation
- Execution-flow analysis
- Feature planning
- File comparison
- AI model discovery
- Local model support
- Prompt/result caching
- Configurable context budgets
- Secret-aware filtering
- Dry-run support

- CI-friendly execution

kontx is actively being developed.

v1 is the foundation, not the finish line.

Roadmap

kontx is still evolving.

Future versions are expected to expand areas such as:

- Deeper language analysis
- More static-analysis rules
- Improved cross-file reasoning
- Incremental project analysis
- Better architecture visualization
- Persistent project context
- IDE integrations
- More CI workflows
- Team workflows
- Additional AI providers and local models

The roadmap will change as the product develops.

Contributing

Issues, ideas and contributions are welcome.

If you find a bug, have a feature request, or want to improve kontx, open an issue or submit a pull request.

License

MIT.

See [LICENSE](#).

Built by **Reabot6**

[Understand the code..](#)